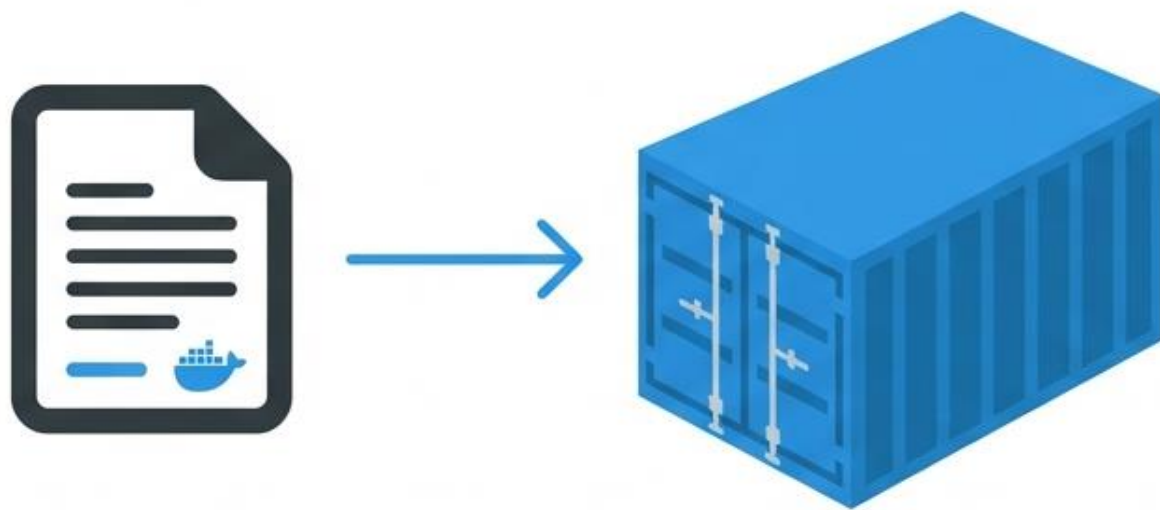


# L'Anatomie d'un Dockerfile

Guide pratique pour la conteneurisation d'applications Python



---

CONCEPTS FONDAMENTAUX & BONNES PRATIQUES

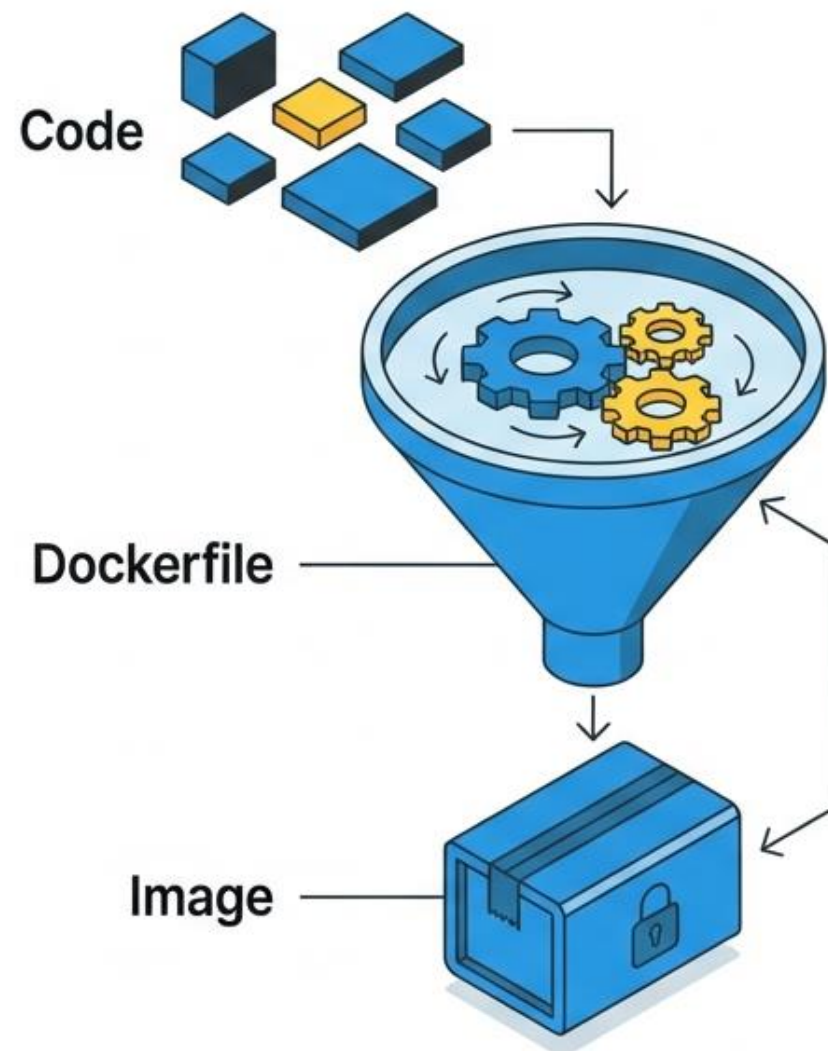
# Qu'est-ce qu'un Dockerfile ?

Un Dockerfile est un simple document texte contenant une série d'instructions ordonnées.

## La Métaphore de la Recette :

- **Ingrédients (Source)** : Code Python, config, dépendances.
- **Instructions (Dockerfile)** : Copier, installer, configurer.
- **Plat Final (Image)** : Artéfact immuable et exécutable.

Automatisation de la création d'image



# L'Instruction FROM : La Fondation

Initialise une nouvelle phase de construction et définit l'image de base.

**FROM** python:3.9-slim

Base Image  
Officielle

Tag Fixe  
(Immutabilité)

Variante Minimale  
(Sécurité/Taille)

## Note Académique

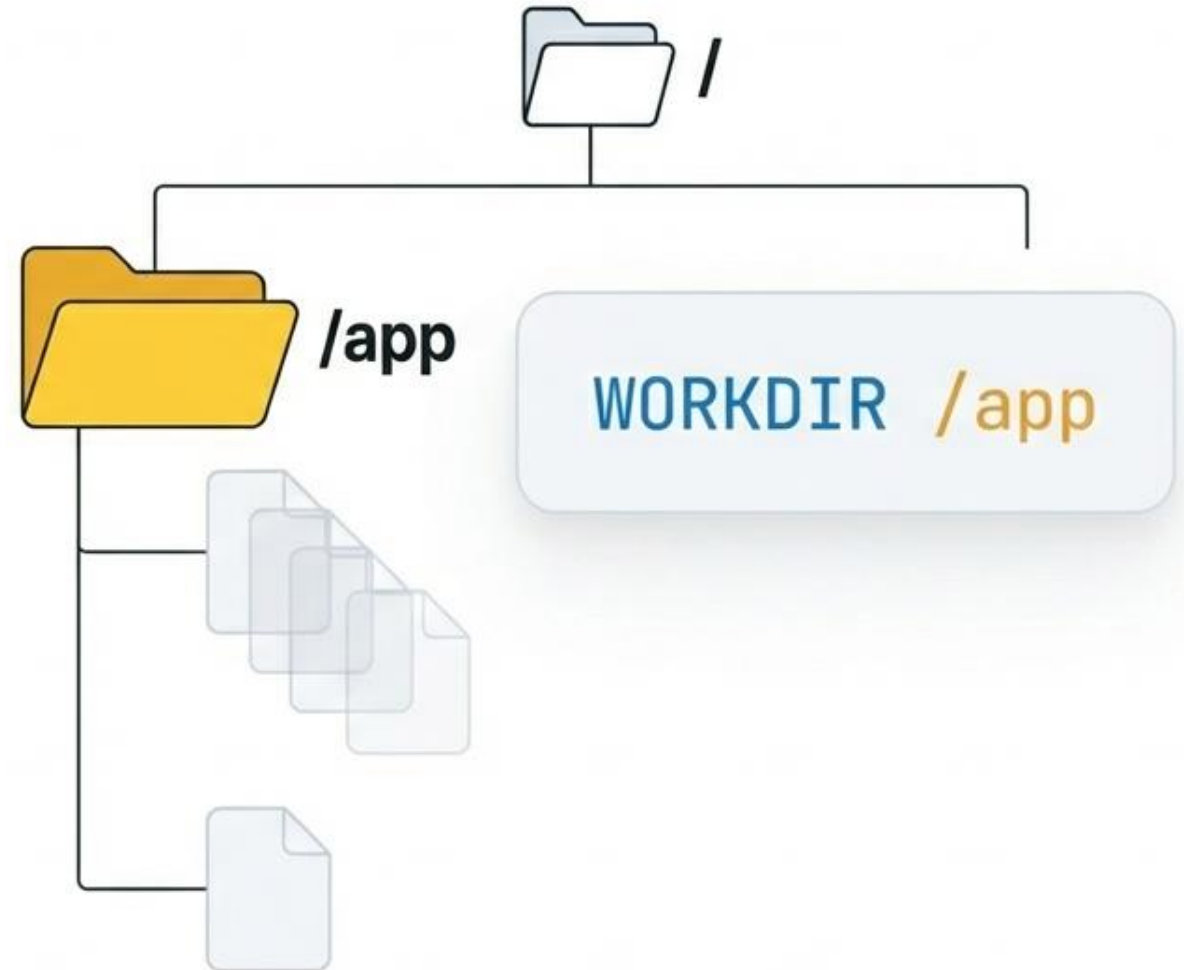
Préférez toujours des images minimales ('slim' ou 'alpine') pour réduire la surface d'attaque et la taille de l'image finale.

# L'Instruction WORKDIR :

## Le Contexte

Définit le répertoire de travail pour toutes les instructions suivantes (RUN, CMD, COPY).

- ☑ Création **automatique** si inexistant.
- ↳ Permet l'usage de chemins relatifs.
- 🛡 **Hygiène** : Évite de polluer la racine du système.





# Ajouter les Ingrédients : COPY vs ADD



**COPY** . .

Copie explicite de fichiers locaux vers le conteneur.

**RECOMMANDÉ**



**ADD** source dest

Peut extraire des archives et télécharger depuis des URLs.

**RISQUE DE SÉCURITÉ**

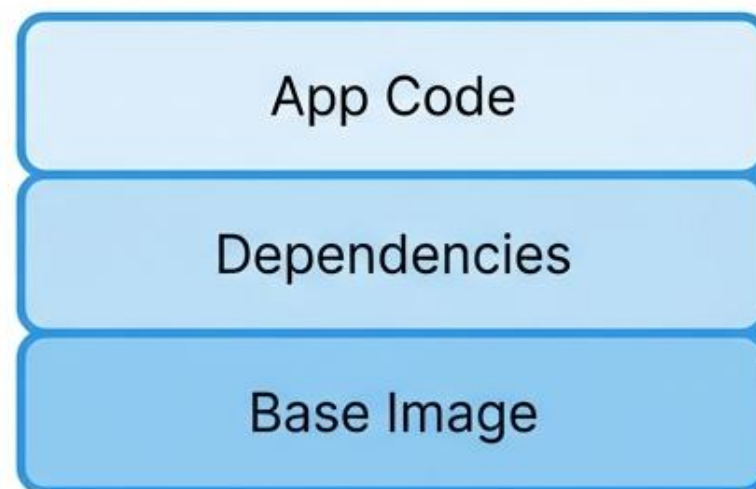
Pourquoi ? ADD peut introduire des risques (Man-in-the-Middle) et des comportements magiques (décompression automatique) non désirés.

# La Phase de Construction : RUN

Exécution de commandes et création de couches (layers).

```
RUN pip install --no-cache-dir -r  
requirements.txt
```

Chaque instruction RUN crée une nouvelle couche immuable.



❗ **Conseil** : Combinez les commandes avec '&&' pour réduire le nombre de couches et la taille finale.

# Configuration : ENV et EXPOSE

ENV PYTHONDONTWRITEBYTECODE=1

Définit des variables d'environnement persistantes.

⚠ ATTENTION : Ne jamais stocker de secrets (mots de passe, clés) dans ENV.

EXPOSE 8000

Documentation réseau.  
Informe Docker que le conteneur écoute sur ce port.



**Documentation**

# Le Service : CMD

La commande de démarrage par défaut.

```
CMD ["python", "main.py"]
```



**Exec Form** (JSON Array) :

- Format **Tableau JSON** : Préféré.
- Avantage : Gestion directe des signaux (SIGTERM) par le processus.

Il ne peut y avoir qu'une seule instruction CMD finale par Dockerfile.



# Synthèse : Le Dockerfile Complet

```
FROM python:3.9-slim
```

— 1. Base (OS + Python)

```
WORKDIR /app
```

— 2. Dossier de travail

```
COPY requirements.txt .
```

— 3. Dépendances (Cache optimisé)

```
RUN pip install --no-cache-dir -r requirements.txt
```

— 4. Installation

```
COPY . .
```

— 5. Code Source

```
EXPOSE 8000
```

— 6. Port Réseau

```
CMD ["python", "main.py"]
```

— 7. Démarrage

# Transformation : La Commande Build

× □ \_

```
$ docker build -t mon-app-python:1.0 .
```

L'ordre au démon  
Docker.

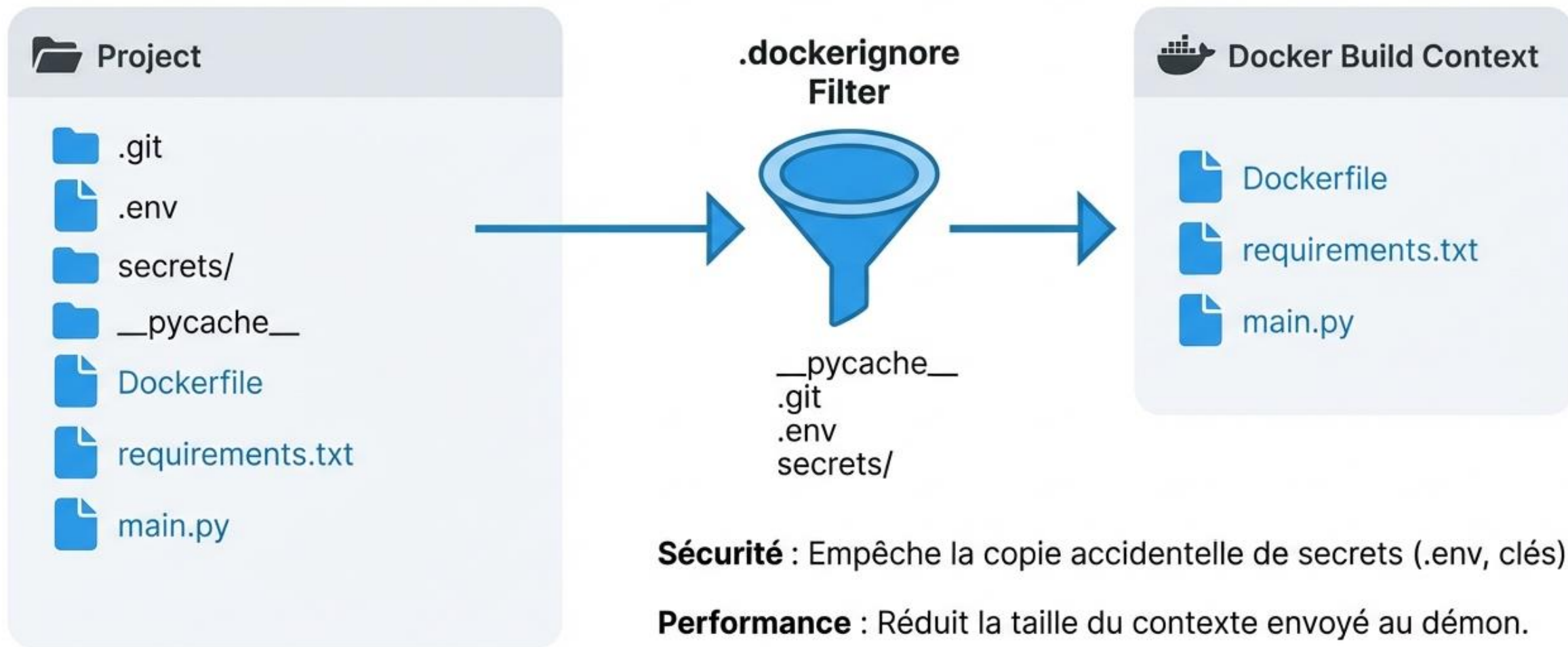
Le Tag  
(Nom + Version).

Le Context  
(Dossier courant).

Docker lit le Dockerfile et exécute les instructions séquentiellement.

# Hygiène : Le fichier .dockerignore

Exclure l'inutile pour la sécurité et la performance.



# Sécurité : L'Utilisateur Non-Root

Principe de moindre privilège.

Par défaut, les conteneurs s'exécutent en tant que ROOT. C'est un risque de sécurité majeur.

```
# Création d'un groupe et utilisateur système
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
# Changement d'utilisateur
USER appuser
```

**"Do Not Root. Do Not Sudo."**



# Gestion des Secrets

**\*\*Règle d'Or\*\*** : Ne stockez jamais de secrets (API Keys, Mots de passe) dans le Dockerfile ou l'image.

**\*\*Pourquoi ?\*\***

L'historique des couches est lisible par tous.



**\*\*Solution Runtime\*\*** :

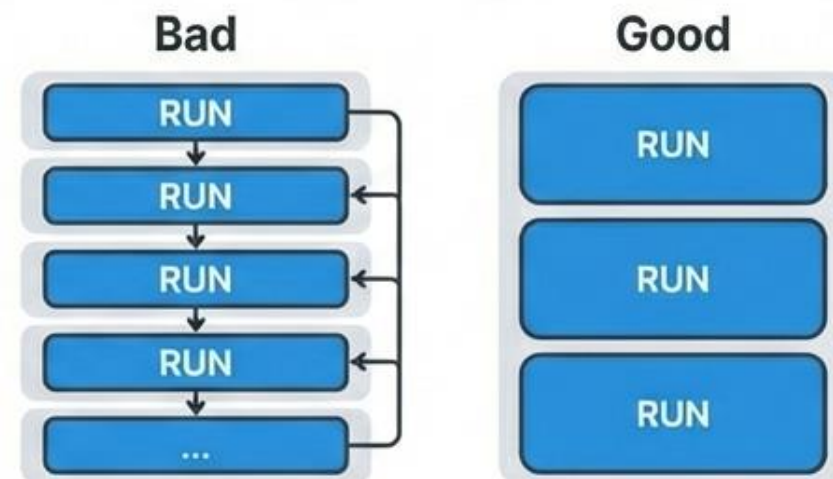
- Variables d'environnement (injectées).
- Montage de volumes (Secrets Manager).

# Optimisation et Validation

## Linting

Utilisez **Hadolint** pour analyser votre Dockerfile et détecter les erreurs.

## Optimisation des Couches



- Minimisez le nombre de couches.
- Groupez les commandes logiques.
- Nettoyez les caches dans la même instruction.

# En Résumé : Les Bonnes Habitudes



**Base Image** : Officielle, spécifique et minimale (slim).



**Instructions** : COPY > ADD. WORKDIR explicite.



**Sécurité** : Utilisateur non-root. Pas de secrets.



**Performance** : .dockerignore et chainage des RUN.

## Ressources

[docs.docker.com/develop/develop-images/dockerfile\\_best-practices/](https://docs.docker.com/develop/develop-images/dockerfile_best-practices/)  
[hadolint.github.io](https://hadolint.github.io)